AGENTIC AI HACKATHON

Hackathon Pack - T09 Legacy Code Documenter & Tribal-Knowledge Q&A

1 · How to Read This Pack

Sections 1–3 help you understand the problem and the person you're solving it for. Section 4 is the contract: what is MANDATORY vs OPTIONAL. Sections 5–6 give you the sample data and a suggested build path so you can start immediately. This same structure is used for every topic - only the content changes.

2 · The Problem

Every mature organisation has at least one codebase that nobody fully understands anymore. The original authors have left; the comments are sparse or wrong; the function names are cryptic abbreviations from a 2014 refactor. When a new developer needs to change a billing calculation or trace why an order cancellation silently fails, they spend days reading code, asking colleagues, and making cautious guesses - because there is no reliable documentation.

This "tribal knowledge" problem is not unique to small teams. Even well-maintained projects accumulate undocumented behaviour over time. The knowledge to answer most questions *is* in the code - it just cannot be queried in plain English, and it cannot cite its source the way a human expert could.

Why it matters

- Maintenance risk: a developer who doesn't understand a function is likely to break it when changing it - with no safety net from the code's own logic.
- Onboarding cost: it takes weeks, not hours, for a new engineer to become productive on an undocumented legacy service.
- Audit and compliance: finance and operations teams regularly need to explain what a billing calculation does - and cannot get a citable answer without reading Python.

3 · Who You're Solving For

Meet your user. Building for a specific person keeps your scope honest.

PERSONA: PIOTR

Name: Piotr - Senior Developer, Application Maintenance

Context: Inherited an order-processing service written in 2019; previous owner left the company; sparse inline comments; zero module-level docs

Goal: Understand what any function does, what it calls, and what side effects it produces - without reading every line of source code

Frustration: *"I asked three colleagues how the discount logic works and got three different answers. I had to trace it myself through four files."*

Success for Piotr: Ask "What does process_order return when all items are out of stock?" and receive a plain-English answer citing the exact file and line number

4 · The Job to Be Done

When Piotr needs to understand or explain a piece of the legacy order-processing codebase, he wants to ask a plain-English question and receive a grounded, cited answer - sourced exclusively from the code itself - so he can act confidently without spelunking through source files or relying on colleagues who may misremember.

THE ONE-SENTENCE TEST

If your demo can answer "What promo codes are accepted and what discount does each give?" with the correct answer and a citation to billing.py, and then correctly refuse to answer "Does this codebase handle payment gateway retries?" because no such code exists - you've nailed the core. Everything else is enhancement.

5 · What You Must Build (and What's Bonus)

This is the contract. Mandatory items are required to qualify for judging. Optional items earn bonus points - only after the mandatory bar is met.

Mandatory (Must Have) - Required to Qualify

1. **Built with Claude** - documentation generation and Q&A answering run through a Claude model.
2. **AST-based parsing** - code units (functions/classes) are extracted from Python files using the ast module, not just raw text chunking.
3. **Generated documentation** - at least orders.py and billing.py must produce a purpose, parameters, and return description for each function.
4. **Cited Q&A** - every answer names the source file and line number it came from.

5. **Grounded refusal** - when the question has no basis in the code, the agent says so rather than guessing.
6. **Working demo** - a live or recorded run answering at least 5 of the 10 sample questions, including the refusal case.
7. **Responsible-AI note** - one line on how you prevent the model from inventing behaviour not present in the code.

Optional (Nice to Have) - Bonus Points

- Dependency graph - visual call graph showing which functions call which, surfaced in the UI.
- Module overview - a 2–3 sentence natural-language summary generated for each file, not just individual functions.
- Accuracy eval - run all 10 questions from questions.csv and report a pass rate against expected_answer_location.
- On-prem mode - switch to a local model (e.g. Bielik via Ollama) so code never leaves the machine - a strong client-trust argument.
- Coverage metric - show what percentage of functions in the repo have generated documentation.
- A simple UI - a searchable doc browser + chat interface a maintenance developer could use daily.

6 · Your Sample Data

You get a small legacy Python repository plus a question set. Start here; you may extend them.

File	Rows / Units	What it is	How to use it
sample_src/orders.py	2 functions	Core order-processing logic: process_order (stock check, reservation, discount, total) and cancel_order; sparse comments; inter-module calls	Parse with ast; document each function; use as retrieval source for Q&A
sample_src/billing.py	2 functions	Financial calculations: calculate_total (VAT) and apply_discount (promo-code table); no docstrings	Primary target for “what discount / tax rate?” questions; contains the refusal test (no retry logic)
sample_src/inventory.py	3 functions	Stock management: check_stock, reserve_item, release_item; called by orders.py; side effects on in-memory STOCK dict	Tests cross-module call tracing: Q&A must attribute answers to inventory.py when asked about stock reservation
questions.csv	10	Plain-English questions about the codebase: question, expected_answer_location (file:function), plus 1 unanswerable refusal case	Your test set and answer key: feed questions to the Q&A; check citations against expected_answer_location; verify the refusal case returns no-basis text

What we suggest adding

This starter set is deliberately small so you can move fast. To make your solution more convincing, consider generating or asking organisers for:

- More files - shipping.py, notifications.py, reports.py to extend the call graph and increase retrieval discrimination.
- Dead code - functions that exist but are never called, to test whether the graph correctly identifies orphans.
- Version history - two versions of billing.py (e.g. old TAX_RATE = 0.22 vs. new 0.23) to test temporal-change reasoning.
- Additional refusal questions - questions about features the codebase genuinely lacks (authentication, retries, logging) so refusal behaviour is stress-tested.

7 · A Suggested Build Path

Not prescriptive - just a route that fits the time-box. Adapt freely.

- Use Python's built-in ast module to parse orders.py and billing.py; extract function names, line ranges, and source snippets. (This is the foundation - reach it in the first 30 minutes.)
- Send a single function snippet to Claude with a prompt asking for purpose, parameters, returns, and gotchas. Confirm you get structured JSON back.
- Build a simple in-memory index of all extracted units. Answer Q01 from questions.csv ("What does process_order return when all items are out of stock?") with a file:line citation.
- Add inventory.py; confirm cross-module questions (Q03 - "What functions are called by process_order?") work correctly.
- Implement grounded refusal: answer Q10 ("Does this codebase handle payment gateway retries?") and verify the model declines.
- Run all 10 questions; record pass/fail against expected_answer_location. Now you have a number to show the jury.
- Pick one bonus item - the dependency graph gives the most visual impact in a 3-minute demo.

8 · Deliverables Checklist

- Working demo (live or recorded) answering >=5 sample questions, each with a file:line citation
- Refusal demonstrated for the unanswerable question (Q10)
- Code or Artifact link
- 3-minute pitch (5+ slides)
- One-line Responsible-AI note (code-grounded answers only)
- (Bonus) pass rate across all 10 questions from questions.csv

- (Bonus) dependency graph or module-level overview